EAST WEST UNIVERSITY

Department of Computer Science and Engineering (CSE)

Project Title

Malware Family Classification Using a Custom Convolutional Neural Network with Attention

Course Code: ICE478

Group: 02

Group Members

Name	Student ID
Sadman Tayef	2022-3-50-002
Shenila Sharior	2022-3-50-009
Mehjabin Tuli	2022-3-50-027
Fihima Tajrian	2023-1-50-006

Table of Contents

Abstract

Malware classification remains a persistent cybersecurity challenge because malware families continuously evolve through repacking and obfuscation, limiting the effectiveness of signature-based detection. This project addresses the problem through image-based malware analysis, where the raw bytes of a malware binary are rendered as a grayscale image and classified using a custom Convolutional Neural Network (CNN) augmented with an attention mechanism. The work was carried out end to end on the Malimg Original dataset (9,339 images, 25 malware families) across three connected stages. First, a thorough exploratory data analysis and a review of six recent CNN- and attention-based malware-classification studies identified a concrete research gap: no prior study isolates the independent contribution of attention from other simultaneous architectural changes. Second, a leakage-free data pipeline was built and four CNN baselines (a compact Simple CNN and three frozen ImageNet-pretrained backbones) were trained, followed by a proposed Custom CNN + CBAM (Convolutional Block Attention Module) model that reuses the Simple CNN backbone unchanged and adds only attention, isolating its effect. The proposed model improved Macro-F1 by 6.64 percentage points over its no-attention counterpart (89.08% - 95.72%) while increasing parameters by only 1.4%, outperforming every pretrained baseline tested. After that the model was further improved through a controlled ablation study, validated with five-fold cross-validation and a paired Wilcoxon significance test against the strongest baseline, and explained using Grad-CAM and LIME on both a correct and an incorrect prediction. The final model reached 93.20% test Macro-F1, and the full three-way comparison, ablation results, and explainability findings are reported and discussed, including an honest account of where the improvement stage did and did not generalize as expected.

Keywords: Malware Classification, Image-Based Malware Detection, Convolutional Neural Network, Attention Mechanism, CBAM, Explainable AI, Grad-CAM, LIME, Ablation Study, Malimg Dataset.

1. Introduction

1.1 Background

Malware classification is a central problem in cybersecurity because malware families continuously evolve through repacking, obfuscation, and code modification, which limits the effectiveness of signature-based detection when new variants no longer match known signatures. Image-based malware analysis offers an alternative representation in which the raw bytes of a malware binary are mapped directly to grayscale pixel intensities, allowing a convolutional neural network to learn recurring texture and structural patterns without requiring full disassembly of the executable. This representation has grown into an active research direction over the last few years, with recent work increasingly combining CNNs with attention mechanisms, cross-modal fusion, and even Vision Transformers to push classification accuracy higher.

1.2 Problem Statement

Although recent CNN- and attention-based malware-image studies report strong accuracy, nearly every one of them introduces attention together with at least one other major change an auxiliary autoencoder, a second input modality, a full Transformer replacing the CNN, a large pretrained backbone, or an additional recurrent branch. As a result, it is not actually possible to tell from the existing literature how much of the reported improvement comes from attention itself, as opposed to the other architectural or data changes introduced alongside it. A second, more practical problem is that image-based malware datasets such as Malimg are known to be severely class-imbalanced and to contain near-duplicate images, both of which can silently inflate reported accuracy if not handled carefully during preprocessing and evaluation. This project addresses both problems together: isolating the true, independent contribution of attention within an otherwise identical CNN, and doing so on a rigorously deduplicated, leakage-free, imbalance-aware data pipeline.

1.3 Research Objectives

- Characterize the Malimg Original dataset through a thorough exploratory data analysis, including class balance, image dimensions, pixel/quality statistics, and data-integrity issues such as corrupt files and duplicate images.
- Build a leakage-free, reproducible data pipeline (deduplication, stratified splitting, and a post-split integrity gate) shared by every subsequent model.
- Train and fairly evaluate four representative CNN baselines (a compact custom CNN and three frozen ImageNet-pretrained backbones).

- Design, implement, and evaluate a custom CNN augmented with a Convolutional Block Attention Module (CBAM), using an identical backbone to the strongest baseline so that any performance change can be attributed specifically to attention.
- Improve the proposed model through a controlled ablation study that varies attention type, network depth/width, regularization, augmentation, and optimizer settings one factor at a time.
- Statistically validate the improvement using five-fold cross-validation on a held-out development set and a paired Wilcoxon significance test against the strongest baseline.
- Explain the final model's decisions using Grad-CAM and LIME on both a correctly and an incorrectly classified example.
- Produce a fair, leakage-free final comparison between the best CNN baseline, the first (Task 2) proposed model, and the final (Task 3) improved model.

1.4 Scope of the Project

The project is scoped to a single public dataset (Malimg Original, grayscale byte images, 25 raw malware families) and a single architectural family (a compact custom CNN with a plug-in attention module), evaluated under one shared, fixed preprocessing and training protocol so that every comparison in this report is apples-to-apples. Attention mechanisms are limited to channel attention, spatial attention, and their combination (CBAM); Vision Transformers, GAN-based augmentation, and multimodal (non-image) inputs all used in parts of the reviewed literature are explicitly outside the scope of this project, since including them would reintroduce the very confound this project sets out to remove. Deployment engineering (e.g., packaging the final model as a production intrusion-detection service) is also outside scope; the deliverable is a validated, explained model and a transparent record of the experiments that produced it.

1.5 Significance and Expected Outcome

The expected outcome is a validated, quantified answer to whether attention helps a compact CNN classify malware-family images, together with a transparent account of how much it helps, how that help was validated statistically, and how the resulting model can be interpreted. Beyond the specific numbers reported, the project is significant methodologically: the leakage-free deduplication pipeline, the matched-backbone attention-isolation design, and the explicit discussion of where the Task 3 improvement stage did and did not generalize are all reusable practices for any future project comparing architectural variants on a small, imbalanced image-classification dataset.

2. Related Works

Six recent studies were reviewed because each applies a CNN, channel or spatial attention, cross-modal attention, or multi-head self-attention to malware-image classification on Malimg or a closely related setting. Each is summarized below together with the specific limitation that, taken collectively, motivates the research gap addressed by this project.

1. Van Dao et al. (2022) [1] proposed a lightweight feature-synthesis pipeline that combines a small CNN with an attention-enhanced variational autoencoder (VAE). The attention component follows CBAM principles, applying channel and spatial attention to refine the learned representation before classification with a conventional classifier; Random Forest produced the best reported accuracy of 99.40% on the unbalanced Malimg dataset under 10-fold cross-validation, falling to 98.40% on a reduced, balanced subset. A key limitation is that this gain cannot be attributed to attention alone, since the pipeline combines CNN features, an attention-VAE representation, and an external classifier rather than comparing the same end-to-end CNN with and without attention.

2. Kim et al. (2023) [2] introduced an attention-based cross-modal CNN that fuses two representations derived from non-disassembled malware binaries: malware images and structural-entropy sequences. Cross-modal attention allows each modality to reinforce the other before higher-level CNN feature extraction, achieving 99.09% accuracy and an F1-score of 0.976 on a filtered Malimg set of 8,250 samples under 10-fold cross-validation. While this shows attention can improve multimodal fusion, its input requirements are more complex than an image-only setting, since the original binaries are needed to derive structural entropy, and the contribution of attention itself is mixed with the added benefit of a second modality.

3. Ben Abdel Ouahab et al. (2023) [3] evaluated a Vision Transformer (ViT) for Malimg classification and compared it against a conventional CNN. The ViT divides each image into patches and applies multi-head self-attention to capture long-range dependencies, reaching 98.38% accuracy against 97.90% for the authors' own CNN baseline. This provides a direct CNN-versus-self-attention comparison, but the ViT is computationally heavier and does not isolate channel attention, spatial attention, and their combination within a single custom CNN backbone, which is the central comparison this project requires.

4. Dong (2024) [4] presented a convolution-free malware-image framework built on opcode-to-RGB conversion, GAN-based synthetic oversampling, and a simplified Vision Transformer classifier using multi-head self-attention to model global relationships

between image patches, achieving 96.97% accuracy and an F1-score of 0.9702 on Malimg. The approach addresses class imbalance and global context together, but because image conversion, GAN augmentation, and self-attention are all introduced simultaneously, the independent effect of attention is difficult to isolate.

5. Ayturan (2026) [5] integrated a Convolutional Block Attention Module (CBAM) after a pretrained ResNet-50 feature extractor, sequentially applying channel and spatial attention to emphasize discriminative malware textures, achieving 99.36% test accuracy on Malimg and 97.25% on the related MaleVis dataset. This demonstrates useful cross-dataset generalization, but the model relies on a large pretrained backbone and targeted augmentation, and does not provide a controlled ablation using the same lightweight custom CNN across no-attention, channel-only, spatial-only, and full-CBAM configurations.

6. Uddin et al. (2026) [6] proposed ParaECA-LSTMNet, combining four parallel CNN branches, Efficient Channel Attention (ECA), additional convolutional refinement, and an LSTM layer, reporting 99.23% accuracy, 99.23% precision, 99.20% recall, and 99.20% F1-score on a stratified 70/15/15 Malimg split. This shows channel recalibration can complement multi-scale and sequential feature modelling, but because several components change simultaneously, the contribution of ECA cannot be isolated cleanly, and the study does not include five-fold cross-validation or significance testing against a matched CNN baseline.

Table 1: Summary of the six related-work studies reviewed for the Custom CNN + Attention track.

Study	Method	Attention Type	Reported Result
Van Dao et al. (2022) [1]	Lightweight CNN + attention-VAE features + Random Forest	CBAM-style channel + spatial	99.40% accuracy (unbalanced Malimg)
Kim et al. (2023) [2]	Cross-modal CNN using images + structural entropy	Cross-modal attention	99.09% accuracy; F1 = 0.976
Ben Abdel Ouahab et al. (2023) [3]	Vision Transformer vs. authors' CNN	Multi-head self-attention	ViT 98.38% accuracy; CNN 97.90%
Dong (2024) [4]	Opcode-to-RGB + GAN oversampling + simplified ViT	Multi-head self-attention	96.97% accuracy; F1 = 0.9702
Ayturan (2026) [5]	Pretrained ResNet-50 + CBAM	Channel + spatial attention	99.36% (Malimg); 97.25% (MaleVis)
Uddin et al. (2026) [6]	ParaECA-LSTMNet (parallel CNN + ECA + LSTM)	Efficient Channel Attention	99.23% accuracy; F1 = 99.20%

Across these six studies, attention consistently contributes to strong reported performance, but in every case it is introduced together with at least one other major architectural or preprocessing change, so its independent contribution is confounded with these other factors. Existing malware-image studies also rarely report class-wise performance under severe imbalance, five-fold cross-validation, paired significance testing, systematic attention ablation, and both Grad-CAM and LIME explainability together in one study. This is the research gap that motivates the present project: a controlled comparison of the same custom CNN backbone without attention, with channel attention, with spatial attention, and with combined channel-spatial (CBAM) attention, validated statistically and explained visually, on a leakage-free version of Malimg.

3. Methodology

The project followed a single, continuous methodology pipeline across all three stages of the work rather than three disconnected procedures: findings from each stage directly shaped the design of the next. Figure 1 summarizes this pipeline end to end, from the initial dataset audit through the baseline and proposed-model experiments to the improvement, validation, and explainability work described in the remainder of this section.

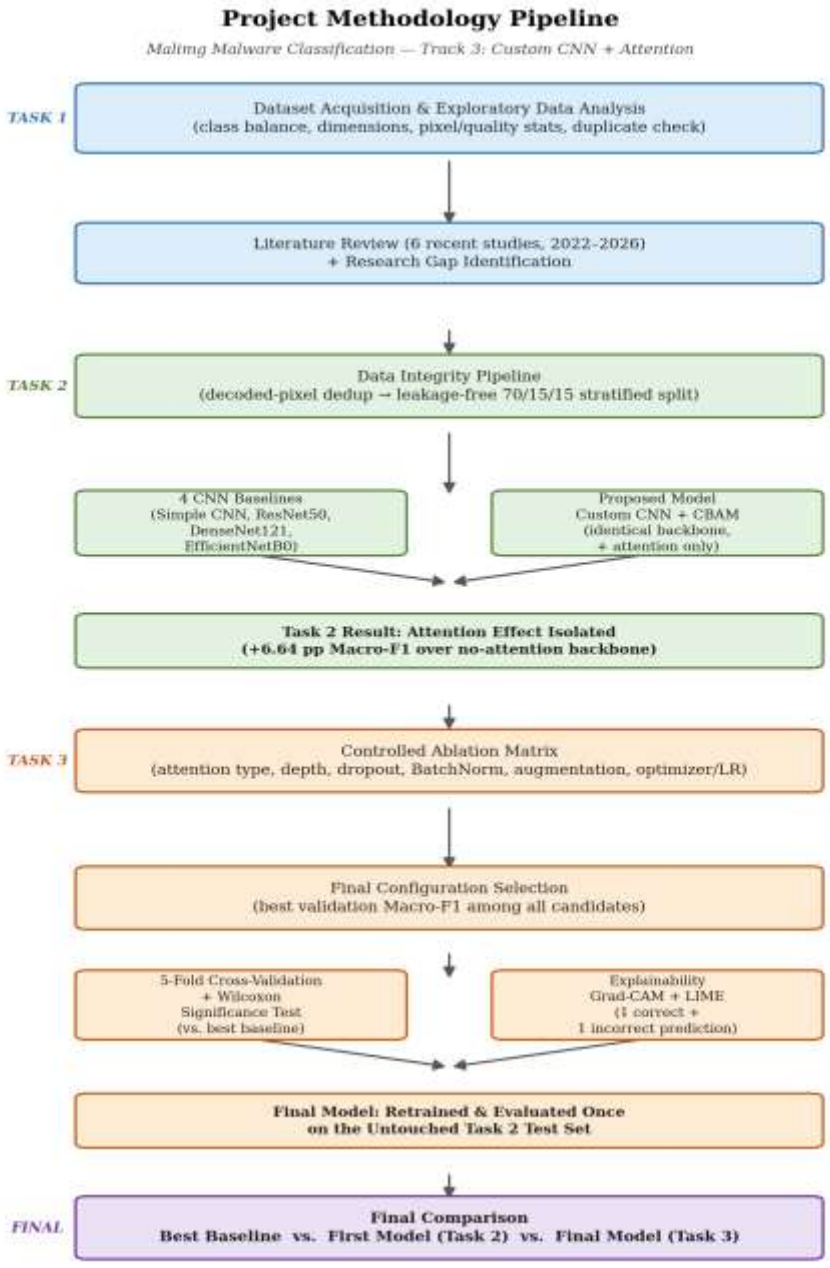

Figure 1. End-to-end project work-flow diagram.

3.1 Dataset Description

The Malimg dataset represents malware binaries as grayscale images, organized by malware family, so that the parent folder name can be used directly as the multiclass label. The canonical Malimg corpus used in this project, sourced from Kaggle [7], contains 9,339 images spanning 25 malware families, stored predominantly as PNG files. A comprehensive exploratory data analysis was carried out first: enumerating and validating every image, extracting dimension, format, and pixel/quality statistics (brightness, contrast, Laplacian variance, Shannon entropy), tabulating class balance, and detecting exact duplicates via SHA-256 hashing.

Table 2: Malimg Original dataset properties.

Property	Description
Dataset name	Malimg Original
Source	Kaggle
Domain	Cybersecurity
Learning task	Multiclass malware-family classification
Raw image count	9,339
Raw number of classes	25
Image representation	Grayscale malware byte images
Dominant file format	PNG

3.2 Data Integrity Pipeline and Leakage-Free Split

The initial duplicate check was repeated at a stricter level: instead of hashing raw file bytes, the fully decoded pixel content of every image was hashed, which catches images that are visually identical but stored with different PNG compression settings. This stricter pass reduced the 9,339 raw files to 8,019 unique decoded images (1,320 redundant copies removed), with zero cross-family duplicate groups. One family, Yuner.A, was found to decode all 800 of its raw files to a single unique image and was therefore excluded from modelling, since it could not support an independent train/validation/test split without leaking that same image across partitions leaving 8,018 images across 24 evaluable families for all subsequent experiments.

The deduplicated images were split 70% / 15% / 15% into training, validation, and test sets using stratified sampling on the class label, so that every one of the 24 families is represented in every partition. Resizing to 224×224 (RGB) was applied inside the data pipeline after this split so that no partition could influence another. A dedicated integrity gate then re-checked, after splitting, that there was zero filepath overlap and zero exact pixel-content-hash overlap between partitions before any model was allowed to train both checks returned zero overlap, confirming the split was leakage-free by construction.

3.3 Preprocessing and Shared Training Protocol

The same preprocessing and training configuration, summarized in Table 3, was reused for every baseline and for the proposed model, and kept as the reference protocol for the improvement stage, so that later comparisons isolate architectural differences rather than pipeline differences.

Table 3: Shared preprocessing and training configuration used for every baseline and the proposed model.

Setting	Value
Input size	224 × 224 × 3
Batch size	32
Split	70% train / 15% validation / 15% test
Augmentation (training only)	Random translation (±2%), zoom (±5%), contrast (±5%)
Imbalance handling	sklearn 'balanced' class weights (training loss only)
Optimizer / loss	Adam / Sparse Categorical Crossentropy
Initial learning rate	1e-3, with ReduceLROnPlateau
Max epochs / early stopping	25 epochs; early stopping on validation loss
Primary metric	Macro-F1 (chosen because of the class imbalance documented above)
Random seed	42 (fixed across NumPy, Python, and TensorFlow)

3.4 Baseline Models

Four representative baselines were trained under the identical pipeline above: a compact custom Simple CNN (three convolutional blocks, global average pooling, a 128-unit dense layer with 0.40 dropout, and a 24-way softmax output), and three ImageNet-pretrained backbones ResNet50 [8], DenseNet121 [9], and EfficientNetB0 [10] used with frozen convolutional weights and only the classification head trained. Freezing the pretrained backbones kept the baseline experiment computationally comparable to training a small network from scratch. Critically, the Simple CNN's backbone and classifier head were reused unmodified as the no-attention counterpart of the proposed CBAM model, which is what makes the attention-effect comparison in Section 4.3 fair.

3.5 Proposed Model: Custom CNN + CBAM

The proposed model directly targets the research gap identified in Section 2: it reuses the exact Simple CNN backbone and classifier head, and inserts a single, well-defined attention module CBAM (Convolutional Block Attention Module) [11] after the last convolutional block, before global average pooling. Because the backbone and classifier are held identical to the no-attention baseline, any performance difference between the two models can be attributed to attention rather than to a confounded architectural change.

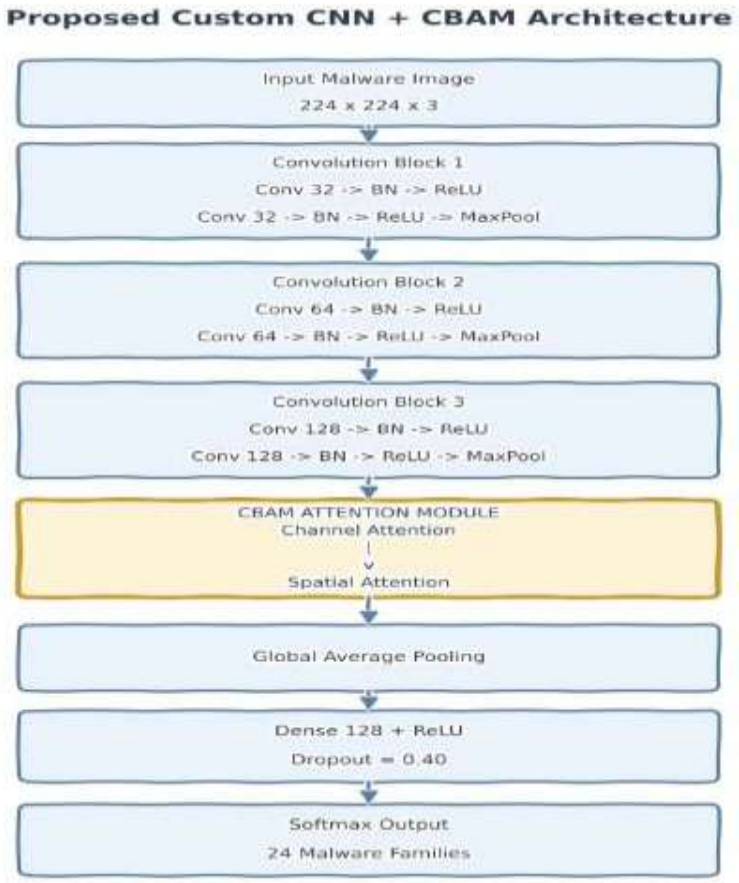

Figure 2: Proposed Custom CNN + CBAM architecture. CBAM is inserted after Convolution Block 3, before global average pooling, so it refines the deepest feature maps before classification.

CBAM was selected because it lets channel and spatial attention be evaluated both jointly and independently (used for the ablation study in Section 3.6), it adds only a small number of parameters, and it is the mechanism used by two of the six related-work studies reviewed in Section 2 , allowing a direct, literature-grounded comparison. Channel attention first determines which of the 128 feature channels coming out of Block 3 matter most, by passing global-average- and global-max-pooled channel descriptors through a shared bottleneck (reduction ratio 8) and combining them with a sigmoid to produce one weight per channel. Spatial attention then determines which regions of the channel-refined feature map matter most, by compressing the channels into an average and a max 2-D map, stacking them, and passing them through a single 7×7 convolution with a sigmoid to produce one weight per spatial location. CBAM applies these steps in sequence channel refinement decides what to look at, spatial refinement decides where to look before the result is pooled and classified.

Table 4: Architecture and training hyperparameters of the proposed Custom CNN + CBAM model.

Setting	Value
Backbone	3 conv blocks (32 → 64 → 128 filters), identical to the Simple CNN baseline
Attention module	CBAM — channel then spatial, inserted after Block 3
Channel reduction ratio	8
Spatial attention kernel	7 × 7
Classifier head	GlobalAveragePooling2D → Dense(128, ReLU) → Dropout(0.40) → Softmax(24)
Total parameters	312,298 (vs. 307,960 for the no-attention Simple CNN — a +1.4% increase)

3.6 Improvement Protocol: Controlled Experiment Matrix

Starting from the Custom CNN + CBAM model as its reference point, this stage ran a controlled ablation matrix that varies one factor at a time rather than an uncontrolled search: attention type (none / channel-only / spatial-only / CBAM), convolutional-block count (2 / 3 / 4), filter width, kernel size (3×3 / 5×5), dropout (0.0 / 0.20 / 0.40 / 0.50), batch normalization (off / on), training-time augmentation (off / on), and optimizer/learning rate (Adam [12] or AdamW at 1e-3 or 5e-4). Every configuration was trained and evaluated on the same locked validation split, and the configuration with the best validation Macro-F1 was carried forward as the final candidate, with validation Macro-Recall used as a supporting metric and a preference for fewer parameters in case of an effective tie.

3.7 Five-Fold Cross-Validation and Significance Testing

To check whether the observed improvement is stable rather than an artifact of one particular split, five-fold cross-validation was performed on the development set (training + validation data combined), while the original test set remained completely untouched throughout tuning and cross-validation. For fairness, both the selected final architecture and the strongest baseline (EfficientNetB0) were evaluated fold-by-fold on the same stratified folds, and the five paired fold-level Macro-F1 values were compared with a Wilcoxon signed-rank test [15] (H0: no systematic paired difference in Macro-F1; H1: a systematic paired difference exists; $\alpha = 0.05$), with the explicit caveat that five folds gives limited statistical power, so a non-significant result should not be read as proof that the two models are equivalent.

3.8 Explainability Protocol

For the final model, one correctly classified test image and one incorrectly classified test image were selected and explained using two complementary techniques: Grad-CAM [13], which produces a coarse heat-map over the last convolutional feature map showing which regions most influenced the predicted class, and LIME [14], which perturbs superpixels of the input and fits a local surrogate model to show which image regions pushed the prediction toward or away from the predicted class. Because Malimg images are byte-derived visual representations rather than natural photographs, the highlighted regions are described throughout this report as discriminative image regions rather than as guaranteed semantic malware structures.

3.9 Evaluation Metrics

Because the dataset is severely imbalanced (Section 4.1), plain accuracy was never used as the sole basis for comparing models. Every model in this project is reported on accuracy, macro precision, macro recall, Macro-F1, and weighted F1, with Macro-F1 adopted as the primary ranking metric throughout, since it weighs every one of the 24 families equally rather than being dominated by the largest classes. Training time, parameter count, model size, and inference time are also reported for every model to capture the efficiency trade-offs alongside predictive performance.

4. Result Analysis

4.1 EDA and Data-Quality Findings

The dataset is highly imbalanced: the largest family, Allaple.A, contains 2,949 images, while the smallest, Skintrim.N, contains only 80 a majority-to-minority ratio of approximately 36.86:1 (Figure 3). Because the two Allaple families alone account for a large share of the dataset, plain accuracy would be dominated by these majority classes; this is why Macro-F1 and class-wise recall were adopted as the primary evaluation emphasis for every model trained in this project (Section 3.9).

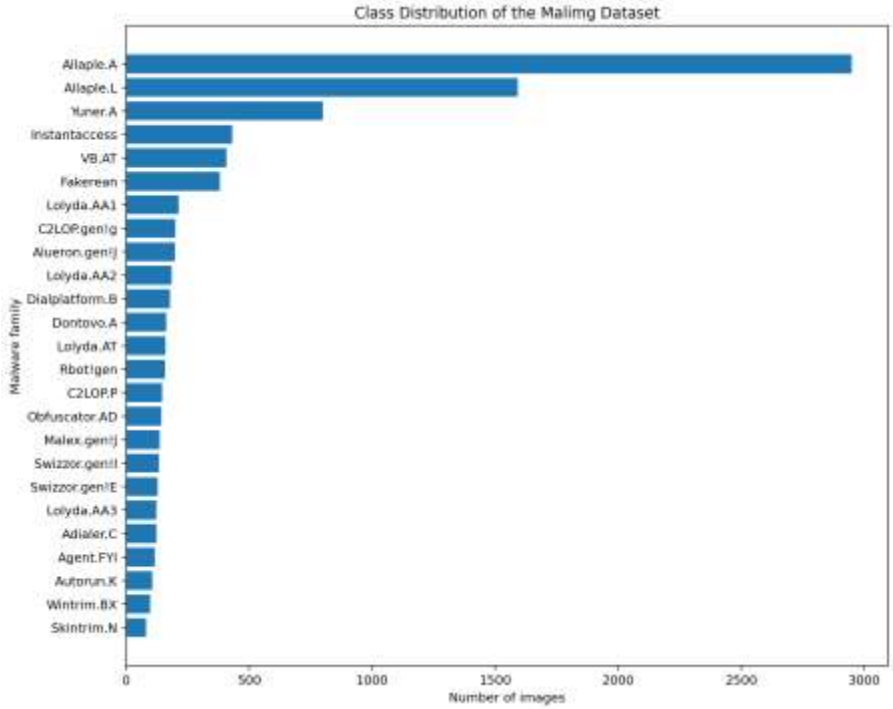

Figure 3: Class distribution of the standard Malimg corpus, showing the 36.86:1 majority-to-minority imbalance between Allaple.A and Skintrim.N.

A labelled grid with one representative image from every family (Figure 4) shows that malware byte images typically contain repeated horizontal bands, dense texture blocks, sparse regions, and family-specific structures; some families are visually distinct while others share similar grayscale textures, which was later confirmed as a source of confusion in the trained models' confusion matrices (Section 4.3).

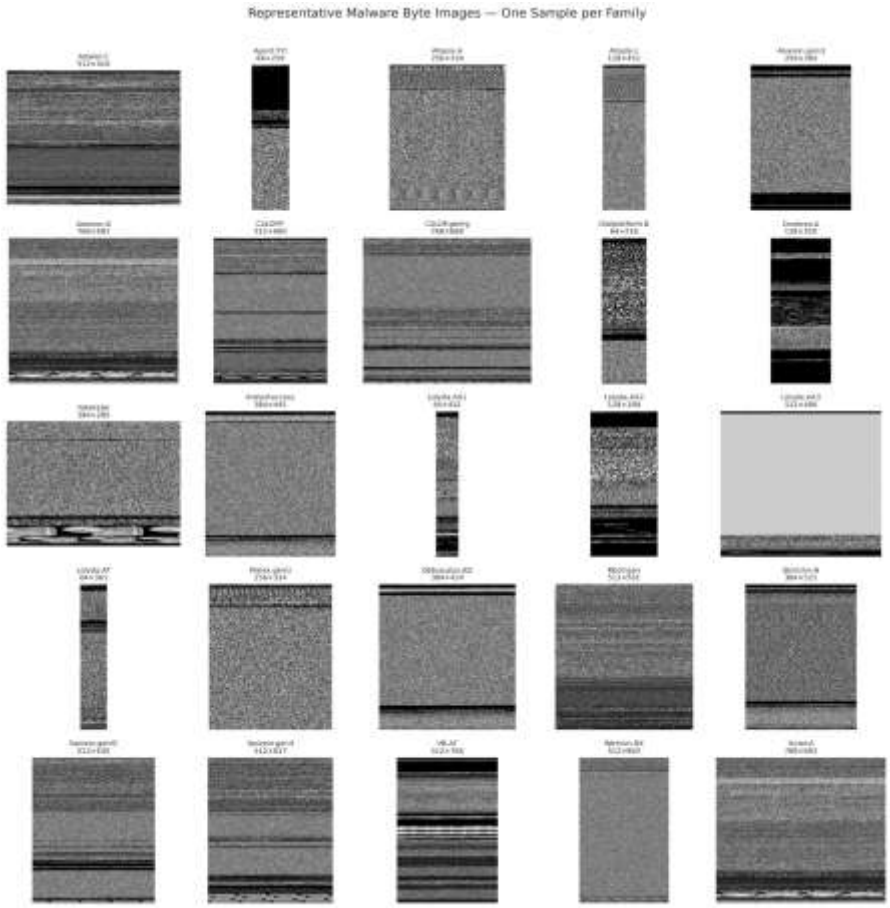

Figure 4: One representative malware byte image from each of the 25 raw Malimg families, illustrating the range of textures, aspect ratios, and structural bands present in the dataset.

Image dimensions varied widely (width 64–1024 px, mean 334.17; height 208–5334 px, mean 436.81; 835 unique resolutions), which is why every model applied the same resizing policy (224×224) after splitting, to ensure a fair comparison. Pixel-level analysis (Figure 5) showed a mean grayscale brightness of 113.57, mean contrast of 79.6, and mean Shannon entropy of 7.226 high enough to reflect the noise-like, non-photographic texture typical of byte-derived images. The data-quality audit found zero corrupt or unreadable images and zero missing metadata, but 260 exact-duplicate file groups (895 redundant copies) using raw byte-level SHA-256 hashing, with zero cross-class duplicates this finding directly motivated the stricter, decoded-pixel-level deduplication carried out afterward (Section 3.2).

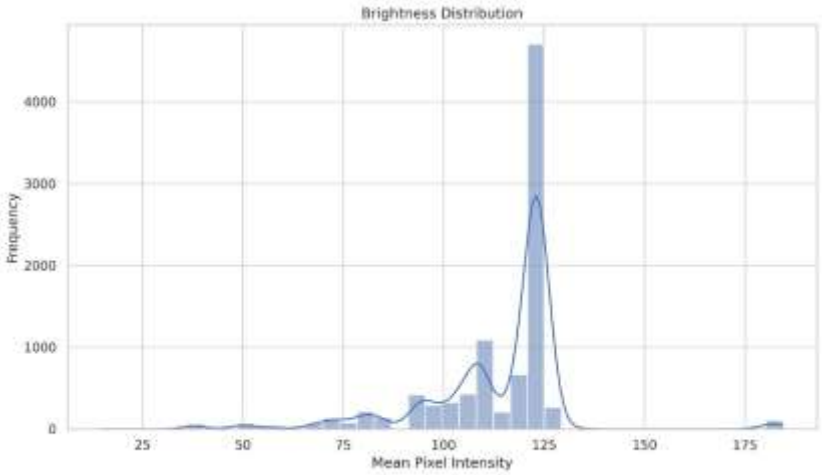

Figure 5: Distribution of mean per-image brightness across the dataset, part of the pixel- and quality-analysis suite (alongside contrast, Laplacian variance, and Shannon entropy).

4.2 Baseline Model Results

Table 5 reports the test-set performance and efficiency of the four CNN baselines, evaluated on the leakage-free, deduplicated 24-family test set.

Table 5: Test-set performance and efficiency of the four CNN baselines.

Model	Accuracy	Macro-F1	Weighted F1	Params	Size (MB)	Train (min)	Infer. (ms/img)
Simple CNN	96.26%	89.08%	95.81%	307,960	3.64	11.45	1.98
ResNet50 (frozen)	94.18%	80.79%	93.64%	23,853,080	93.67	2.95	7.59
DenseNet121 (frozen)	96.18%	91.88%	95.90%	7,171,800	29.85	10.13	11.18
EfficientNetB0 (frozen)	96.26%	92.96%	96.39%	4,216,635	18.18	5.43	6.02

EfficientNetB0 was the strongest baseline by Macro-F1 (92.96%), followed by DenseNet121 (91.88%) and the lightweight Simple CNN (89.08%), with frozen ResNet50 last (80.79%). The gap between accuracy and Macro-F1 was largest for ResNet50 (13.4 points) and smallest for EfficientNetB0 (3.3 points), confirming that ResNet50's high raw accuracy was disproportionately propped up by the majority classes, while its ImageNet-tuned filters built for natural-image edges, colours, and object parts transferred poorly onto the synthetic, texture-only statistics of malware byte images.

4.3 Proposed Model Results and the Attention Effect

The proposed Custom CNN + CBAM model achieved 98.50% accuracy, 95.24% macro precision, 96.90% macro recall, 95.72% Macro-F1, and 98.50% weighted F1 on the test set, training in 13.77 minutes over 17 epochs with 312,298 parameters, a 3.70 MB model size, and 2.91 ms/image inference making it simultaneously the best-performing and the most parameter-efficient model evaluated at this stage.

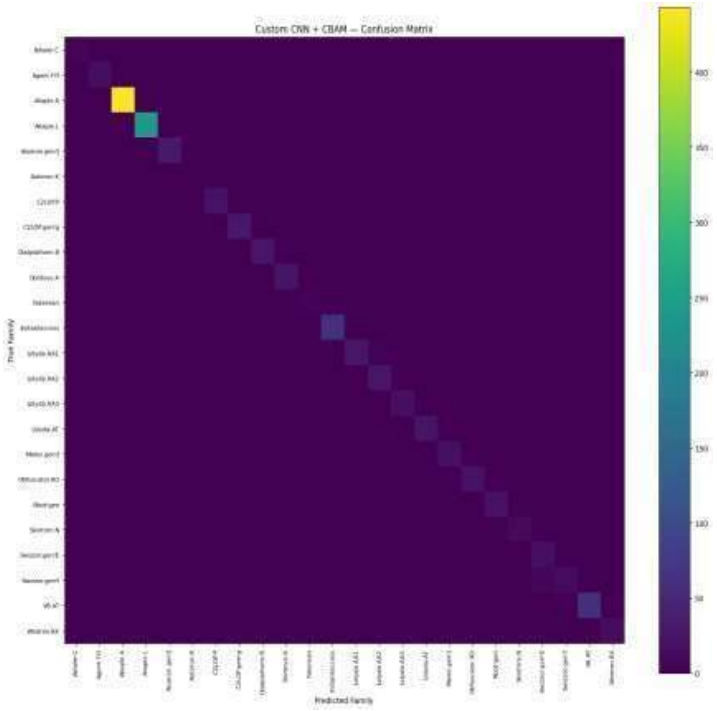

Figure 6: Confusion matrix of the Custom CNN + CBAM model on the held-out test set. The matrix is almost entirely diagonal across all 24 families.

Because the proposed model reuses the Simple CNN backbone unchanged and adds only CBAM, the difference between the two results isolates the effect of attention itself (Table 6).

Table 6: Direct attention effect Simple CNN (no attention) vs. Custom CNN + CBAM, identical backbone.

Metric	Simple CNN (no attention)	Custom CNN + CBAM	Change
Accuracy	96.26%	98.50%	+2.24 pp
Macro Recall	92.29%	96.90%	+4.61 pp
Macro F1	89.08%	95.72%	+6.64 pp

Adding CBAM increased total parameters by only 1.4% yet produced the single largest improvement of any model change at this stage (+6.64 percentage points of Macro-F1), with capacity held almost constant the strongest direct evidence that attention, not extra capacity, was responsible for the gain. Across all five models compared so far (Figure 7), Custom CNN + CBAM ranked first by Macro-F1, ahead of EfficientNetB0 (the best baseline) by 2.76 percentage points, while using roughly 13× fewer parameters.

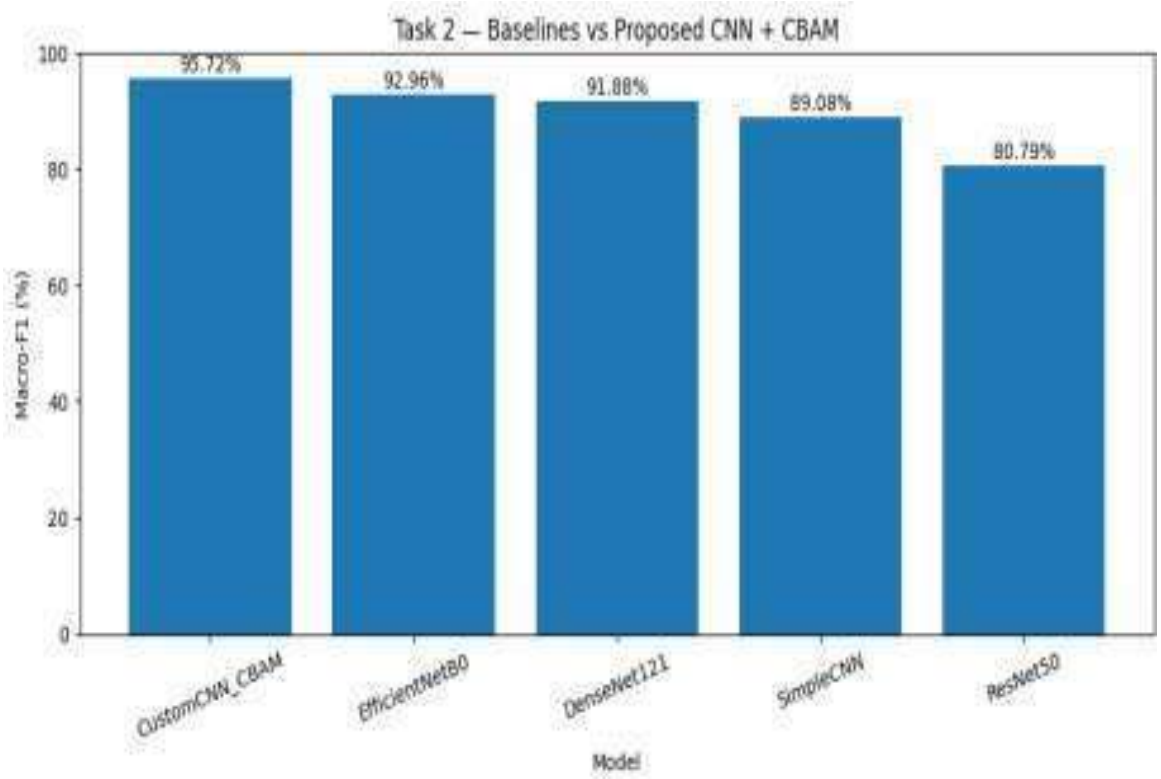

Figure 7: Macro-F1 of all four baselines and the proposed Custom CNN + CBAM model.

Two limitations were carried forward into the improvement stage: all numbers so far came from a single train/validation/test split, so the reported gains had not yet been checked for statistical stability across resampled splits; and a small number of visually similar families (notably Swizzor.gen!E / Swizzor.gen!I) remained a persistent source of confusion even with attention.

4.4 Ablation and Improvement Results

Table 7 reports the top validation - Macro-F1 configurations from the controlled experiment matrix described in Section 3.6 (validation set, development split).

Table 7: Top validation Macro-F1 configurations from the controlled improvement experiment matrix.

Run	Attention	Key change	Val. Macro-F1	Val. Macro-Recall	Val. Accuracy
blocks_4	CBAM	4 conv blocks (32-64-128-256)	97.35%	97.25%	97.59%
augmentation_off	CBAM	Training-time augmentation disabled	97.25%	97.39%	98.92%
ablation_spatial	Spatial only	Spatial attention only	97.01%	96.83%	98.92%
adam_5e4	CBAM	Adam, LR 5e-4	96.98%	96.96%	98.75%
dropout_050	CBAM	Dropout 0.50	96.68%	96.40%	98.75%
adamw_1e3	CBAM	AdamW, LR 1e-3	96.24%	96.41%	98.34%
first_model_cbam	CBAM	Reference configuration (3 blocks)	95.62%	96.04%	97.76%
filters_large	CBAM	64-128-256-512 filters	94.92%	95.07%	98.01%
ablation_channel	Channel only	Channel attention only	94.77%	95.20%	97.26%
dropout_020	CBAM	Dropout 0.20	93.87%	93.99%	97.67%
dropout_0	CBAM	Dropout 0.00	93.65%	94.74%	97.17%
ablation_cnn	None	No-attention backbone	93.53%	94.13%	96.59%
filters_small	CBAM	16-32-64-128 filters	93.16%	93.25%	96.51%
batchnorm_off	CBAM	BatchNorm disabled	92.20%	94.56%	92.19%
kernel_5x5	CBAM	5×5 conv kernel	92.10%	93.00%	96.59%

The best validation Macro-F1 (97.35%) was obtained by deepening the backbone to four convolutional blocks (blocks_4); the resulting configuration CBAM attention, four blocks (32-64-128-256 filters), 3×3 kernels, batch normalization on, dropout 0.40, augmentation on, Adam at 1e-3 was selected as the final configuration and carried forward to five-fold cross-validation and final test evaluation.

Table 8 summarizes the ablation results, expressed as the change in validation Macro-F1 relative to the reference configuration.

Table 8: Ablation results change in validation Macro-F1 relative to the reference CBAM configuration.

Ablation factor	Configuration	Val. Macro-F1	Δ vs. reference (pp)
Depth	4 conv blocks (CBAM)	97.35%	+1.73
Augmentation	Off (CBAM)	97.25%	+1.64
Attention type	Spatial only	97.01%	+1.39
Dropout	0.50 (CBAM)	96.68%	+1.07
Attention type	Channel only	94.77%	-0.85
Dropout	0.20 (CBAM)	93.87%	-1.74
Dropout	0.00 (CBAM)	93.65%	-1.97
Attention type	None (no attention)	93.53%	-2.08
Normalization	BatchNorm off (CBAM)	92.20%	-3.42
Depth	2 conv blocks (CBAM)	91.29%	-4.33
Attention type	CBAM (reference config, repeat run)	85.20%	-10.41

Most ablation results are consistent with the central finding of Section 4.3: removing attention entirely (ablation_cnn) costs 2.08 points of Macro-F1 relative to the CBAM reference, and disabling batch normalization or shrinking the network to two blocks costs even more. One repeat run of the reference CBAM configuration, however, converged to a markedly lower score (85.20% Macro-F1, 69.08% validation accuracy) than every other CBAM run in the matrix, including the no-attention baseline. Given that an architecturally identical configuration reached 95.6%+ Macro-F1 elsewhere in the same matrix (first_model_cbam) and in the fully executed earlier experiment (95.72% on test), this single run is best interpreted as training instability an unlucky random initialization settling into a poor local minimum during the noisy early epochs that class-weighted training is known to produce rather than as evidence that CBAM itself is harmful; it is reported here for completeness and transparency rather than excluded from the table.

4.5 Cross-Validation and Significance Test Results

Table 9 reports the five-fold cross-validation summary (mean ± standard deviation) on the development set for the selected final model and for the EfficientNetB0 baseline.

Table 9: Five-fold cross-validation summary (mean ± standard deviation) on the development set.

Model	Macro-F1 (mean ± std)	Macro-Recall (mean ± std)	Accuracy (mean ± std)
Final Model	95.15% ± 1.27	96.04% ± 1.14	97.14% ± 1.25
EfficientNetB0 (baseline)	92.51% ± 1.80	93.99% ± 1.35	96.43% ± 0.90

The final model's cross-validated Macro-F1 (95.15%) exceeds EfficientNetB0's (92.51%) by 2.64 points on average across the five folds. A paired Wilcoxon signed-rank test on the five fold-level Macro-F1 values gave a statistic of 1.0 and $p = 0.125$, which is above the $\alpha = 0.05$ significance threshold the difference is directionally consistent across folds but is not statistically significant at this sample size. With only five paired observations, statistical power is inherently limited, so this non-significant result should be read as inconclusive rather than as evidence that the two models perform equivalently.

4.6 Final Model Test Performance

After model selection, the final configuration was retrained from scratch on the locked training and validation partitions and evaluated exactly once on the untouched test set. The final model reached 97.59% accuracy, 93.52% macro precision, 95.33% macro recall, 93.20% Macro-F1, and 97.51% weighted F1, with 1,227,898 parameters, a 14.20 MB model size, 10.26 minutes of training time, and 3.60 ms/image inference.

4.7 Final Comparison: Best Baseline vs. First Model vs. Final Model

Table 10 places the best baseline, the first proposed CBAM model (Section 4.3), and the final improved model side by side on the same untouched test set.

Table 10: Final comparison best baseline vs. first model vs. final model.

Model	Accuracy	Macro Prec.	Macro Recall	Macro-F1	Weighted F1	Params	Size (MB)
Best baseline - EfficientNetB0	96.26%	92.33%	94.79%	92.96%	96.39%	4,216,635	18.18
First model - Custom CNN + CBAM	98.50%	95.24%	96.90%	95.72%	98.50%	312,298	3.70
Final model - improved CBAM	97.59%	93.52%	95.33%	93.20%	97.51%	1,227,898	14.20

Two observations stand out from Table 10. First, all three models comfortably exceed the accuracy/Macro-F1 achieved by three of the four original baselines (ResNet50, DenseNet121, Simple CNN), confirming that attention continues to provide a real benefit over the no-attention and pretrained alternatives even after the improvement stage. Second, and more importantly, the final model (93.20% Macro-F1) does not exceed the first model (95.72% Macro-F1) on this untouched test set, even though the four-block configuration that produced the final model was selected specifically because it had the highest validation Macro-F1 (97.35%) among all fifteen configurations tried, and uses roughly four times as many parameters. This gap between a strong validation score and a comparatively weaker test score is the classic signature of validation-selection sensitivity: choosing the single best-scoring configuration out of many candidates from one non-cross-validated run per candidate tends to reward whichever configuration happened to fit that particular validation split best, which is not guaranteed to be the configuration that generalizes best to unseen data. This limitation is discussed further in Section 5.

4.8 Explainability Results

Grad-CAM and LIME were applied to one correctly classified test image (true and predicted class: Allaple.A) and one misclassified test image (true class Rbot!gen, predicted as Autorun.K).

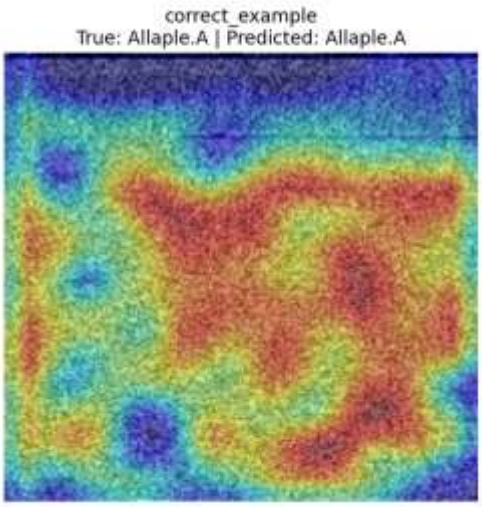

Figure 8a: Grad-CAM, correctly classified Allaple.A example.

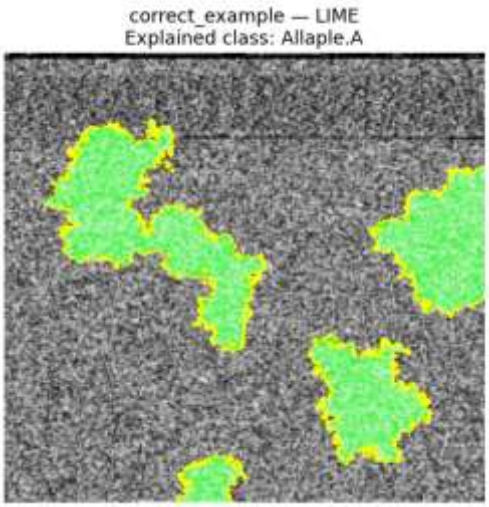

Figure 8b: LIME, correctly classified Allaple.A example.

For the correctly classified Allaple.A image, Grad-CAM highlights a broad, connected mid-to-lower region of dense texture rather than the whole image, showing the model relied on a specific structural band rather than every pixel equally. LIME's superpixel explanation agrees in direction, marking several clustered green (supporting) regions concentrated in the same general texture area, which is consistent with the family-distinguishing horizontal bands and dense texture blocks that the labelled-image analysis (Section 4.1, Figure 4) had already flagged as visually informative.

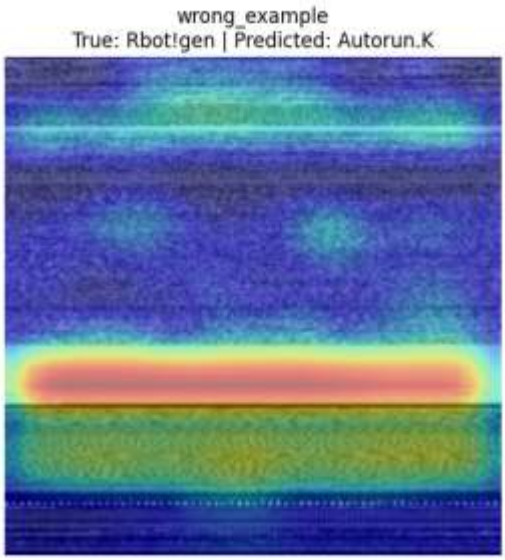

Figure 9a. Grad-CAM, misclassified example true: Rbot!gen, predicted: Autorun.K.

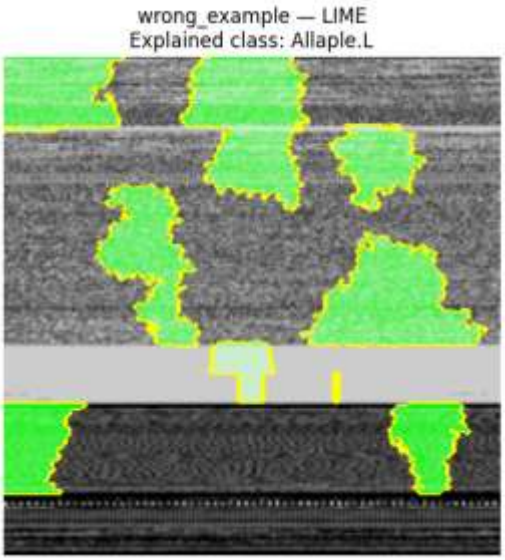

Figure 9b. LIME, misclassified example true: Rbot!gen, predicted: Autorun.K.

For the misclassified Rbot!gen image (predicted as Autorun.K), Grad-CAM concentrates almost all attention on a single narrow horizontal band, essentially ignoring the rest of the image, and LIME's supporting superpixels are similarly clustered around a few horizontal strips near the top and bottom of the image rather than spread across a distinguishing texture region. This suggests the model over-relied on a generic horizontal-band pattern that appears in several families rather than on texture specific to Rbot!gen, offering a plausible, image-level explanation for why this particular error occurred and pointing to horizontal-band-dominated families as a useful target for future error analysis.

5. Discussion

Taken together, the three stages of this project provide a clear picture of the dataset and modelling problem. The initial EDA showed that Malimg is severely imbalanced (36.86:1) and contains many duplicate images. This motivated the use of Macro-F1 as the primary metric and stricter decoded-pixel-level deduplication before modelling. Without this step, the 800 duplicate Yuner.A images could have caused data leakage across the train, validation, and test sets. The related-work review showed that previous Malimg studies did not clearly isolate the contribution of attention from other architectural changes. This was addressed by keeping the backbone and classifier head identical between the no-attention Simple CNN and the proposed CBAM model. CBAM improved Macro-F1 by 6.64 percentage points with only a 1.4% increase in parameters. This strongly suggests that attention, rather than additional model capacity, drove the improvement. The ablation study supported this finding. Removing attention, disabling batch normalization, or reducing network size lowered validation Macro-F1. Channel-only and spatial-only attention also achieved smaller gains than full CBAM, supporting its channel-then-spatial design.

However, model selection revealed an important limitation. The final model was chosen as the single best validation result from fifteen candidates, with only one training run per candidate. The selected four-block model achieved 97.35% validation Macro-F1 but only 93.20% on the untouched test set, compared with 95.72% for the smaller first model. Despite having roughly four times more parameters, the larger model generalized worse. This indicates sensitivity to the validation split and possible over-selection from noisy validation estimates. The anomalously low CBAM ablation run also suggests meaningful run-to-run variation.

Five-fold cross-validation provided a more robust comparison. The final model achieved a mean Macro-F1 of, compared with for EfficientNetB0. Although this suggests that attention remains beneficial, the difference was not statistically significant ($p = 0.125$). Therefore, the result should be considered suggestive rather than conclusive. Future model selection should evaluate candidates using cross-validation and multiple training runs. A simpler alternative would be a tolerance-based rule, such as selecting the smallest model within one percentage point of the best validation score. Several three-block CBAM models were already close to the best model while using roughly one-quarter of its parameters.

Finally, the explainability results provide qualitative support for the quantitative findings. For the correctly classified example, Grad-CAM and LIME both focused on a broad, structured texture region, suggesting that the model learned meaningful discriminative features. In the misclassified example, both methods focused mainly on a narrow horizontal band, suggesting reliance on a generic pattern shared across families. Such cases provide a useful target for future model and dataset refinement.

6. Conclusion

This project carried a single research question does attention genuinely help a custom CNN classify malware-family images, and can that benefit be isolated, validated, and explained rather than merely observed through three connected stages on the Malimg Original dataset. The initial data-understanding stage established the dataset's severe class imbalance, dimension variability, and duplicate-image issues, and identified a clear gap in prior literature: no existing study isolates attention's contribution from other simultaneous architectural changes. The baseline-and-proposed-model stage closed that gap directly, building a leakage-free pipeline and showing that adding CBAM to an otherwise unchanged compact CNN backbone improved Macro-F1 by 6.64 percentage points (89.08% → 95.72%) while increasing parameters by only 1.4%, outperforming every pretrained baseline tested, including EfficientNetB0, DenseNet121, and ResNet50.

The improvement stage confirmed the core attention finding under a wider set of controlled ablations and cross-validated it against the strongest baseline, finding a directionally consistent but not statistically significant advantage at five folds (95.15% vs. 92.51% Macro-F1, $p = 0.125$). However, the specific final configuration selected by taking the single best validation score among many candidates a deeper, four-block network did not generalize better than the original proposed model on the untouched test set (93.20% vs. 95.72% Macro-F1), despite using substantially more parameters. This outcome is itself an informative finding rather than a failure of the project: it demonstrates concretely why model selection from many single-run candidates needs either cross-validated re-evaluation of the leading candidates or an explicit preference for smaller models within a tolerance of the best score, and it leaves the original, smaller Custom CNN + CBAM model as the strongest validated candidate produced across the full project. Combined with the Grad-CAM and LIME explanations, which showed the model attending to plausible, family-relevant texture regions on a correct prediction and a diffuse, generic band on an incorrect one, this project delivers not only a high-performing attention-augmented malware classifier but also a transparent account of where it succeeds, where it fails, and why directly answering the research gap identified at the outset.

7. References

[1] T. Van Dao, H. Sato, and M. Kubo, “An Attention Mechanism for Combination of CNN and VAE for Image-Based Malware Classification,” IEEE Access, vol. 10, pp. 85127-85136, 2022. doi: 10.1109/ACCESS.2022.3198072.

[2] J. Kim, J.-Y. Paik, and E.-S. Cho, “Attention-Based Cross-Modal CNN Using Non-Disassembled Files for Malware Classification,” IEEE Access, vol. 11, pp. 22889-22903, 2023. doi: 10.1109/ACCESS.2023.3253770.

[3] Ben Abdel Ouahab, L. Elaachak, and M. Bouhorma, “Enhancing Malware Classification with Vision Transformers: A Comparative Study with Traditional CNN Models,” Proceedings of NISS 2023, Larache, Morocco, 2023.

[4] H. Dong, “Convolutional-Free Malware Image Classification Using Self-Attention Mechanisms,” Informatics and Automation, vol. 23, no. 6, pp. 1869-1898, 2024. doi: 10.15622/ia.23.6.11.

[5] K. Ayturan, “A Hybrid Deep Learning Approach with Spatial Attention Mechanism for Visual-Based Malware Detection in IoT Security,” Black Sea Journal of Engineering and Science, vol. 9, no. 3, pp. 1256-1268, 2026. doi: 10.34248/bsengineering.1851726.

[6] R. Uddin et al., “Cybersecurity Oriented Malware Classification Using ParaECA-LSTMNet with a Hybrid Attention Guided CNN-LSTM Framework,” Discover Analytics, vol. 4, article 2, 2026. doi: 10.1007/s44257-025-00053-2.

[7] Malimg Original, Kaggle dataset. Available: https://www.kaggle.com/datasets/ikrambenabd/malimg-original.

[8] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778.

[9] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, “Densely Connected Convolutional Networks,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2017, pp. 4700-4708.

[10] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in Proc. Int. Conf. Machine Learning (ICML), 2019, pp. 6105-6114.

[11] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, “CBAM: Convolutional Block Attention Module,” in Proc. European Conf. Computer Vision (ECCV), 2018, pp. 3-19.

[12] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in Proc. Int. Conf. Learning Representations (ICLR), 2015.

[13] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2017, pp. 618-626.

[14] M. T. Ribeiro, S. Singh, and C. Guestrin, “‘Why Should I Trust You?’: Explaining the Predictions of Any Classifier,” in Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2016, pp. 1135-1144.

[15] F. Wilcoxon, “Individual Comparisons by Ranking Methods,” Biometrics Bulletin, vol. 1, no. 6, pp. 80-83, 1945.